Laporan Proyek Berpikir Komputasional (WI1102)

Kelas K05



Disusun Oleh:

Jovan Nathaniel Permana (16026295)
Qamila Annur Firdausi (16026380)
Abira Mutia Salsabila (16026060)
Ezra Hadyan Wicaksono (16026055)

Fakultas FMIPA Institut Teknologi Bandung

Semester 1 Tahun 2026/2027

1 Dasar-dasar Berpikir Komputasional yang Digunakan

1.1 Algoritma Newton-Raphson

1.1.1 Dekomposisi

Algoritma Newton-Raphson sebenarnya adalah algoritma optimasi yang bersifat iteratif, dengan tujuan utama untuk mencari akar-akar/titik potong sebuah fungsi $f(x)$ dengan sumbu x. Dalam proyek ini, algoritma Newton-Raphson difokuskan untuk mencari akar-akar dari sebuah polinom dengan derajat tertentu, sesuai masukan dari pengguna (*user*). Melalui prinsip dekomposisi (pemecahan suatu masalah dengan skala besar menjadi skala yang lebih kecil), algoritma Newton-Raphson untuk mencari akar-akar sebuah fungsi polinom dapat dipecah menjadi beberapa tugas berikut.

1. Menginput derajat polinom (N), nilai koefisien-koefisiennya, serta nilai tebakan awal (x_0) dari user.
2. Untuk nilai tebakan awal $x = x_0$ yang diberikan, hitung nilai fungsi $f(x_0)$ beserta turunannya, yaitu $f'(x_0)$.
3. Nilai x diperbarui secara terus-menerus melalui prosedur iteratif dengan persamaan
$$x_{n+1} \leftarrow x_n - \frac{f(x_n)}{f'(x_n)} \tag{1}$$
hingga nilai *error* yang terjadi dalam penentuan akar menjadi tidak signifikan atau berada di bawah ambang batas tertentu.
4. Jika $f'(x) = 0$ pada suatu tahap tertentu atau perulangan pada langkah 3 tidak dapat selesai (*infinite loop*), maka proses perhitungan akar gagal dilakukan.

1.1.2 Pengenalan Pola

Algoritma Newton-Raphson memiliki pola yang mirip dengan algoritma optimasi, yaitu algoritma yang menekankan perhitungan matematika secara iteratif (berulang) untuk mencapai suatu tujuan. Dalam algoritma Newton-Raphson, persamaan (1) dijalankan secara berulang-ulang untuk mencapai suatu tujuan, yaitu akar persamaan polinom. Selama perulangan tersebut, nilai x akan terus diperbarui (*update*) hingga error dalam pencarian akar menjadi tidak signifikan atau lebih kecil dari ambang batas tertentu.

1.1.3 Abstraksi

Algoritma Newton-Raphson membutuhkan beberapa informasi penting, yaitu:

1. Derajat polinom N.
2. Koefisien-koefisien pada polinom.
3. Nilai tebakan awal x_0.

Dalam proyek ini, derajat polinom N disimpan di dalam variabel bertipe `int`, sementara nilai tebakan awal x_0 disimpan di dalam variabel bertipe `float`. Nilai-nilai koefisien pada polinom disimpan di dalam sebuah array `float` yang berukuran $N + 1$.

1.2 Algoritma Pinjam Buku

1.2.1 Dekomposisi

1.2.2 Pengenalan Pola

1.2.3 Abstraksi

1.3 Algoritma Labirin

1.3.1 Dekomposisi

1.3.2 Pengenalan Pola

1.3.3 Abstraksi

2 Lampiran

2.1 Lampiran 1: Kode Python Tanpa Menggunakan Subprogram

2.1.1 Algoritma Newton-Raphson

Kode 1: Kode Python algoritma Newton-Raphson tanpa subprogram

```
1 import time
2 """ Newton Raphson (tanpa subprogram) """
3
4 # program utama
5 # input
```

```
N = int(input('Masukkan derajat polinomial: '))
N += 1
coefs = [0]*N
for i in range(N):
    if(i == 0):
        val = float(input('Masukkan nilai konstanta: '))
    else:
        val = float(input(f'Masukkan nilai koefisien x^{i}: '))

    coefs[i] = val

x = float(input('Masukkan tebakan awal x: '))
epsilon = 1e-4
k = 0
max_iter = 1e6
# proses iterasi
start_time = time.perf_counter()
while True:
    k += 1
    val = 0.0
    el = 0
    element = 1
    while el <= (N-1):
        val += coefs[el] * (x ** el)
        el += 1
    grad = 0.0
    while element <= (N-1):
        grad += element * coefs[element] * (x ** (element - 1))
        element += 1

    if grad == 0:
        print('Newton-Raphson gagal: turunan = 0')
        break

    x_new = x - (val / grad)
    print(f'Iterasi ke-{k} --> Nilai x: {x_new:.6f} Gradien: {grad:.6f}')
    delta = x_new - x
    if delta < 0:
        delta = -delta

```

```
    if val > 0:
        absval = val
    else:
        absval = -val

    if (delta < epsilon) and (absval < epsilon):
        print(f'Akar ditemukan: {x_new:.6f}')
        print(f'Jumlah iterasi: {k}')
        break
    elif k >= max_iter:
        print(f'\nGAGAL: Akar tidak ditemukan setelah {k} iterasi.')
        break

    x = x_new

end_time = time.perf_counter()
execution_time = end_time - start_time
print(f"Waktu eksekusi: {execution_time:.6f} detik")
```

2.1.2 Algoritma Pinjam Buku

Kode 2: Kode Python algoritma Pinjam Buku tanpa subprogram

```
"""Pinjam Buku (Tanpa Subprogram)"""
```

2.1.3 Algoritma Labirin

Kode 3: Kode Python algoritma Labirin tanpa subprogram

```
"""labirin (tanpa Subprogram)"""
```

2.2 Lampiran 2: Kode Python Dengan Menggunakan Subprogram

2.2.1 Algoritma Newton-Raphson

Kode 4: Kode Python algoritma Newton-Raphson dengan subprogram

```
import time
""" Newton Raphson (dengan subprogram) """

```

```
# definisi fungsi
def function(x, coefs):
    value = 0.0
    for i in range(len(coefs)):
        value += coefs[i] * (x**i)
    return value

def gradient(x, coefs):
    value = 0.0
    for i in range(1, len(coefs)):
        value += i * coefs[i] * (x**(i - 1))
    return value

def absol(x):
    value = (x >= 0) * x + (x < 0) * (-x)
    return value

# program utama
# input
N = int(input('Masukkan derajat polinomial: '))
N += 1
coefs = [0]*N
for i in range(N):
    if(i == 0):
        val = float(input('Masukkan nilai konstanta: '))
    else:
        val = float(input(f'Masukkan nilai koefisien x^{
            i}: '))

    coefs[i] = val

x = float(input('Masukkan tebakan awal x: '))
epsilon = 1e-4
k = 0
max_iter = 1e6
# proses iterasi
start_time = time.perf_counter()
while True:
    k += 1
    val = function(x, coefs)
    grad = gradient(x, coefs)
    if grad == 0:
        print('Newton-Raphson gagal: turunan = 0')
```

```
        break

    x_new = x - (val / grad)
    print(f'Iterasi ke-{k} --> Nilai x: {x_new:.6f}
       Gradien: {grad:.6f}')

    if (absol(x_new - x) < epsilon) and (absol(val) <
       epsilon):
        print(f'Akar ditemukan: {x_new:.6f}')
        print(f'Jumlah iterasi: {k}')
        break
    elif k >= max_iter:
        print(f'\nGAGAL: Akar tidak ditemukan setelah {k
           } iterasi.')
        break

    x = x_new

end_time = time.perf_counter()
execution_time = end_time - start_time
print(f"Waktu eksekusi: {execution_time:.6f} detik")
```

2.2.2 Algoritma Pinjam Buku

Kode 5: Kode Python algoritma Pinjam Buku dengan subprogram

```
"""Pinjam Buku (dengan Subprogram)"""
```

2.2.3 Algoritma Labirin

Kode 6: Kode Python algoritma Labirin dengan subprogram

```
"""labirin (dengan Subprogram)"""
```